

Greedy Algorithms

SMITA SANKHE

Assistant Professor

Department of Computer Engineering

3/10/2023



Problem Characteristics

- Decomposable?
- Solution steps be undone?
- Unique solution or set of solutions?
- Any solution or best solution? •

Solution –a path or a state

3/10/2023



2

Greedy Algorithms

- typically apply to optimization problems in which we make a set of choices in order to arrive at an optimal solution.

- make each choice in a locally optimal manner.
- Does not give optimal solutions always; but do most of the times
- Quite powerful and works well for a wide range of problems.

3/10/2023



The greedy method

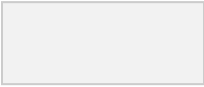
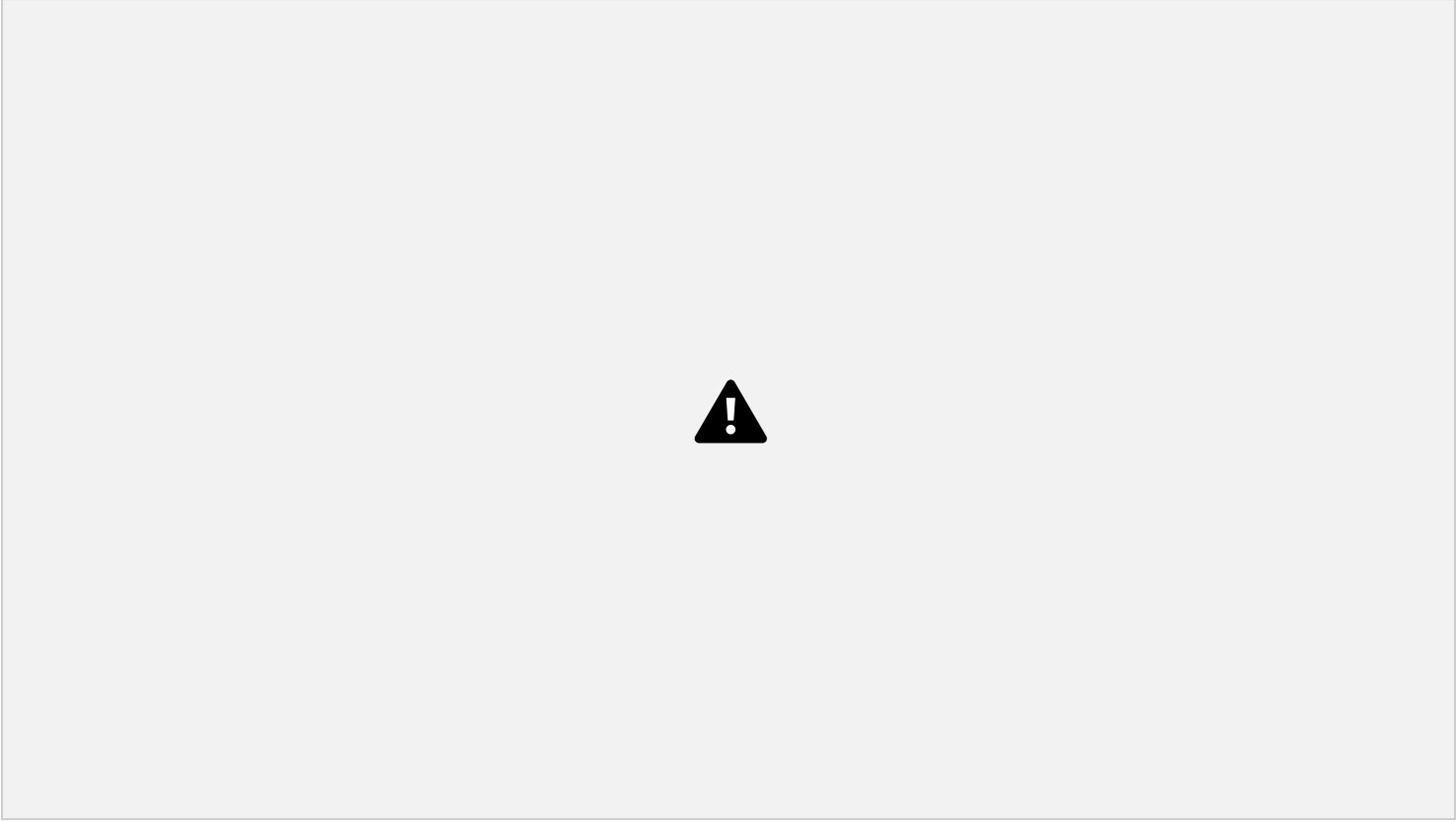
- Suppose that a problem can be solved by a sequence of decisions. The greedy method has that each decision is locally optimal. These locally optimal solutions will finally add up to a globally optimal solution.
- Only a few optimization problems can be solved by the greedy method.

3/10/2023

4 -4



The general method





Greedy algorithms: The working

- Make a choice at each step.
- Make the choice before solving the subproblems.
- Solve top-down.

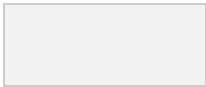


6

Problems typically solved with Greedy Strategy

- Knapsack problem

- Minimum cost spanning trees-Kruskal and prim's algorithm
- Single source shortest path
- Job sequencing with deadlines



Knapsack problem

- Given positive integers $P_1, P_2, \dots, P_n, W_1, W_2, \dots, W_n$ and M .
- Find $X_1, X_2, \dots, X_n, 0 \leq X_i \leq 1$ such that

is maximized.

$$\sum_{i=1}^n P_i X_i$$

$=$

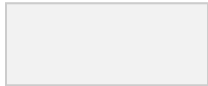
$$\sum_{i=1}^n W_i X_i$$

$$W_i X_i M \leq$$

- Subject to

n

4 -8



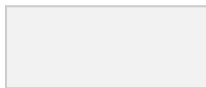
Knapsack Problem Example

- $M = 20, (P_1, P_2, P_3) = (25, 24, 15)$
 $(W_1, W_2, W_3) = (18, 15, 10)$

- Three feasible solutions, 1 is optimal

(X_1, X_2, X_3)	$\Sigma W_i X_i$	$\Sigma P_i X_i$
$(1, 2/15, 0)$	20	
$(0, 2/3, 1)$	20	

1. 28.2 2. 31 3. $(0, 1, 1/2)$ 20 31.5



Applications of - Knapsack Problem

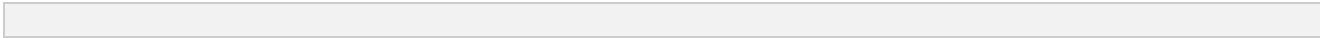
- finding the least wasteful way to cut raw materials,
- seating contest of [investments](#) and [portfolios](#), • seating contest of assets for [asset-backed securitization](#),
- generating keys for the [Merkle–Hellman](#) and other [knapsack cryptosystems](#)
- construction and scoring of tests in which the test takers have a choice as to which questions they answer

- Resource optimization



Minimum spanning trees (MST)

- $G = (V, E)$: weighted connected undirected graph
- Spanning tree : $S = (V, T)$, $T \subseteq E$, undirected tree
- Minimum spanning tree(MST) : a spanning tree with the smallest total weight.



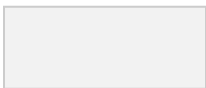
4 -11





An example of MST

- A graph and one of its minimum costs spanning tree



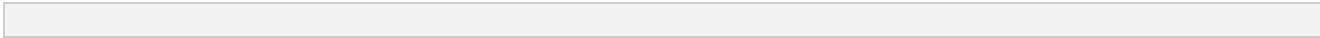
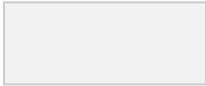


Kruskal's algorithm for finding MST

Step 1: Sort all edges into nondecreasing order.

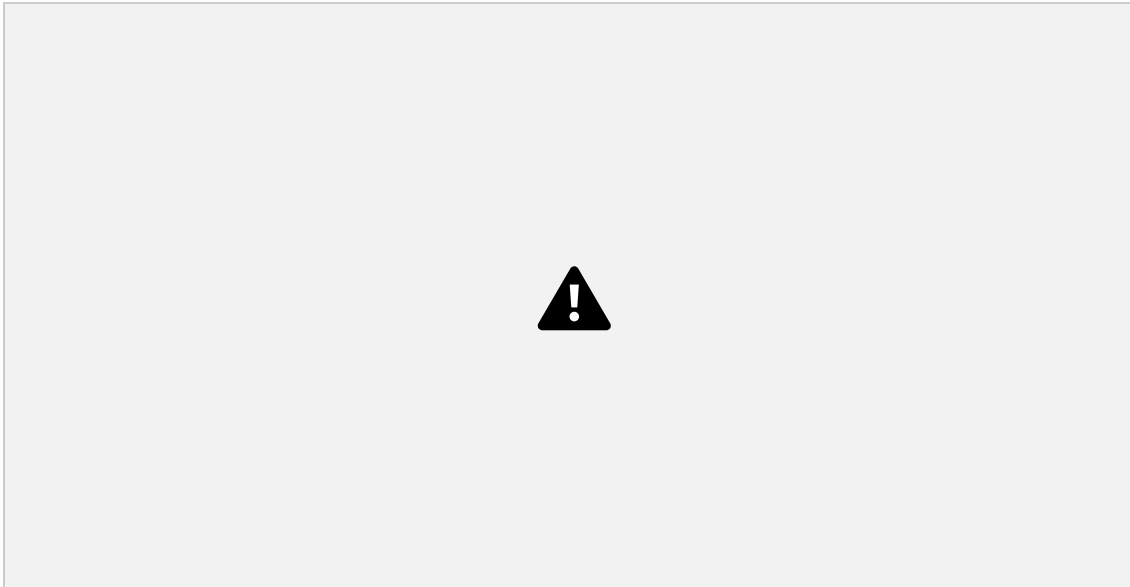
Step 2: Add the next smallest weight edge to the forest if it will not cause a cycle.

Step 3: Stop if $n-1$ edges. Otherwise, go to Step2.



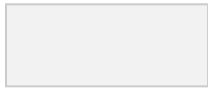
4 -13





An

example of Kruskal's algorithm

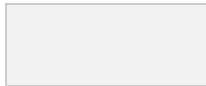


The details for constructing MST

- How do we check if a cycle is formed when a new edge is added?
 - By the SET and UNION method.
- A tree in the forest is used to represent a SET.
- If $(u, v) \in E$ and u, v are in the same set, then the

addition of (u, v) will form a cycle.

- If $(u, v) \in E$ and $u \in S_1, v \in S_2$, then perform UNION of S_1 and S_2 .



4 -15



Prim's algorithm for finding MST

Step 1: $x \in V$, Let $A = \{x\}$, $B = V - \{x\}$. Step

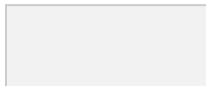
2: Select $(u, v) \in E$, $u \in A$, $v \in B$ such that (u, v) has the smallest weight between A and B .

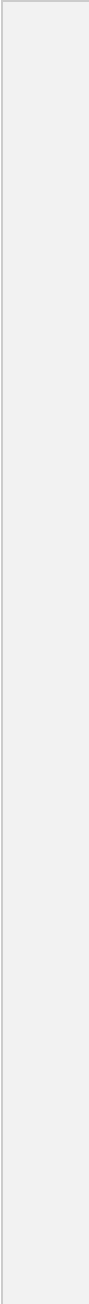
Step 3: Put (u, v) in the tree. $A = A \cup \{v\}$, $B = B - \{v\}$

Step 4: If $B = \emptyset$, stop; otherwise, go to Step 2.

- Time complexity : $O(n^2)$, $n = |V|$.

(see the example on the next page)





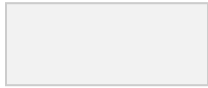
An example for Prim's algorithm



Applications of - MST

- **Network design.**
 - *telephone, electrical, hydraulic, TV cable, computer, road*
- **Approximation algorithms for NP-hard problems.** – [traveling salesperson problem](#), [Steiner tree](#)
- **Indirect applications.**
 - max bottleneck paths
 - codes for error correction
 - learning salient features for real-time face verification –
 - reducing data storage in sequencing amino acids in a protein –
 - model locality of particle interactions in turbulent fluid flows –
 - autoconfig protocol for Ethernet bridging to avoid cycles in a network
- [Circuit design](#): implementing efficient multiple constant ~~multiplications~~, as used in [finite impulse response](#) filters. •

Regionalisation of socio-geographic areas, the grouping of areas into ~~homogeneous~~, contiguous regions.



18



Single Source Shortest Path

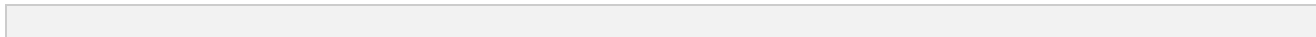
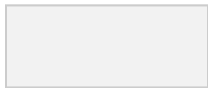
- Problem Definition

Given a directed graph $G = \langle V, E \rangle$, a weighting function cost for edges of G and a source vertex v_0 , the problem is to compute the shortest paths from v_0 to all the

remaining vertices of G .

Assumption:- all weights are positive.

Shortest path between v_0 to some other node v is an ordering among a subset of edges.





Single Source Shortest Path Conditions

- Feasibility condition:- All possible paths with source vertex i.e a. $n-1$ possible paths and
b. possible iterations $n-2$ excluding source and destination vertex
- Optimal condition :- path that includes all vertices with minimum cost



21

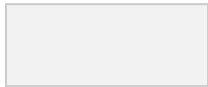
Complexity

- Overall operations-
= $N \text{ vertices} * (n-2)$

$$\text{iterations} = n(n-2)$$

$$= n^2 - 2n$$

$$= O(n^2)$$



Applications of - SSSP

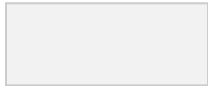
- automatically find directions between physical locations, such as driving directions on [web mapping](#) websites
- When the problem is represented as a FSA, can be used to find an optimal sequence of choices to reach a certain goal state, or to establish lower bounds on the time needed to reach a given state.
- In a networking or telecommunications mindset, this shortest path problem is sometimes called the min-delay path problem and usually tied with a [widest path problem](#)
- plant and facility layout, [robotics](#), [transportation](#), and [VLSI](#) design



Applications of - SSSP

- Maps
- Robot navigation.
- Texture mapping.
- Urban traffic planning.
- Optimal pipelining of VLSI chip.
- Subroutine in advanced algorithms.
- Telemarketer operator scheduling.

- Routing of telecommunications messages.
- Approximating piecewise linear functions.
- Network routing protocols (OSPF, BGP, RIP).
- Exploiting arbitrage opportunities in currency exchange.
- Optimal truck routing through given traffic congestion pattern



~~Dijkstra's algorithm~~



Cost adjacency matrix.
All entries not shown
are $+\infty$.

1 2 3 4 5 6 7 8

1 0

2 300 0

3 1000 800 0

4 1200 0

5 1500 0 250

6 1000 0 900 1400

7 0 1000



Vertex

Iteration S Selected (1) (2) (3) (4) (5) (6) (7) (8) Initial ----

1	5	6	$+\infty$	$+\infty$	$+\infty$	1500	0	250	$+\infty$	$+\infty$	2	5,6	7	$+\infty$	$+\infty$	$+\infty$	1250	0	250	1150	1650
3	5,6,7	4	$+\infty$	$+\infty$	$+\infty$	1250	0	250	1150	1650	4	5,6,7,4	8	$+\infty$	$+\infty$	2450	1250	0	250		
	1150	1650	5	5,6,7,4,8	3	3350	$+\infty$	2450	1250	0	250	1150	1650	6	5,6,7,4,8,3	2					
			3350	3250	2450	1250	0	250	1150	1650	5,6,7,4,8,3,2	3350	3250	2450	1250	0					
																	250	1150	1650		

- Time complexity: $O(n^2)$

4-26

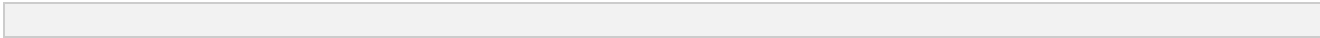
The single-source shortest path
problem • shortest paths from v_0 to all
destinations



4 -27

SSSP Solution

- Problem definition
- Feasibility & Optimality Condition
- OR
- Explain how the answer will be optimal
- Compute
- Final answer
 - Cost of every destination
 - Path to every destination
- complexity



28

